

Oops Experiment - 9

Name - Nidhi Kesharwani
Sapld - 500120314
Batch - 4

1) Store a list of integers in an `ArrayList<Integer>` using autoboxing. Iterate through the list, unbox each value, and determine if it is a prime number, printing the results.

```
import java.util.ArrayList;

public class PrimeChecker {
    public static void main(String[] args) {
        // Creating an ArrayList of Integer (autoboxing)
        ArrayList<Integer> numbers = new ArrayList<>();

        // Adding some numbers using autoboxing
        numbers.add(10);
        numbers.add(7);
        numbers.add(23);
        numbers.add(15);
        numbers.add(29);
        numbers.add(44);
        numbers.add(2);

        // Iterating through the list and checking for prime numbers
        for (Integer num : numbers) {
            int value = num; // Unboxing the Integer to int
            if (isPrime(value)) {
                System.out.println(value + " is a prime number.");
            } else {
                System.out.println(value + " is not a prime number.");
            }
        }
    }

    // Method to check if a number is prime
    public static boolean isPrime(int num) {
        if (num < 2) return false;
        for (int i = 2; i <= Math.sqrt(num); i++) {
            if (num % i == 0) {
                return false;
            }
        }
        return true;
    }
}
```

```
}  
}
```

```
10 is not a prime number.  
7 is a prime number.  
23 is a prime number.  
15 is not a prime number.  
29 is a prime number.  
44 is not a prime number.  
2 is a prime number.
```

2) Create an ArrayList to store Employee objects with attributes like name, id, and salary. Add three employees, update one employee's salary, remove another by their id, and print the remaining employees.

```
import java.util.ArrayList;  
import java.util.Iterator;
```

```
public class Main { // Online compilers usually require the main class to  
be named 'Main'
```

```
    // Employee class inside the same file
```

```
    static class Employee {  
        private int id;  
        private String name;  
        private double salary;
```

```
        // Constructor
```

```
        public Employee(int id, String name, double salary) {  
            this.id = id;  
            this.name = name;  
            this.salary = salary;  
        }
```

```
        // Getters
```

```
        public int getId() {  
            return id;
```

```

    }

    public String getName() {
        return name;
    }

    public double getSalary() {
        return salary;
    }

    // Setter for salary
    public void setSalary(double salary) {
        this.salary = salary;
    }

    // Override toString() for easy printing
    @Override
    public String toString() {
        return "Employee{id=" + id + ", name='" + name + "', salary=" +
salary + "'}";
    }
}

public static void main(String[] args) {
    // Create an ArrayList to store Employee objects
    ArrayList<Employee> employees = new ArrayList<>();

    // Adding three employees
    employees.add(new Employee(101, "Alice", 50000));
    employees.add(new Employee(102, "Bob", 55000));
    employees.add(new Employee(103, "Charlie", 60000));

    // Update salary of employee with ID 102 (Bob)
    for (Employee emp : employees) {
        if (emp.getId() == 102) {
            emp.setSalary(60000); // Updating salary
            break;
        }
    }
}

```

```

// Remove employee with ID 101 (Alice)
Iterator<Employee> iterator = employees.iterator();
while (iterator.hasNext()) {
    if (iterator.next().getId() == 101) {
        iterator.remove();
        break;
    }
}

// Print remaining employees
System.out.println("Remaining Employees:");
for (Employee emp : employees) {
    System.out.println(emp);
}
}
}

```

```

Remaining Employees:
Employee{id=102, name='Bob', salary=60000.0}
Employee{id=103, name='Charlie', salary=60000.0}

```

- 3) Use a HashMap to manage a product inventory where keys are productId (Integer) and values are quantity (Integer). Add three products, update the quantity of one product, remove another, and display the final inventory.

```

import java.util.HashMap;
import java.util.Map;

public class ProductInventory {
    public static void main(String[] args) {
        // Creating a HashMap to store product inventory (productId ->
        quantity)
        HashMap<Integer, Integer> inventory = new HashMap<>();

        // Adding three products
    }
}

```

```

inventory.put(101, 50); // Product ID 101 with quantity 50
inventory.put(102, 30); // Product ID 102 with quantity 30
inventory.put(103, 20); // Product ID 103 with quantity 20

// Update the quantity of product with ID 102
inventory.put(102, 40); // New quantity for product 102

// Remove the product with ID 101
inventory.remove(101);

// Display the final inventory
System.out.println("Final Inventory:");
for (Map.Entry<Integer, Integer> entry : inventory.entrySet()) {
    System.out.println("Product ID: " + entry.getKey() + ", Quantity: "
+ entry.getValue());
}
}

```

```

Final Inventory:
Product ID: 102, Quantity: 40
Product ID: 103, Quantity: 20

```

- 4) Given an array of names with duplicates (e.g., ["Aman", "Varchasv", "Sourabh", "Varchasv", "Aman"]), store them in a HashSet to eliminate duplicates. Check if a specific name exists in the set and print the unique names.

```

import java.util.HashSet;

public class UniqueNames {
    public static void main(String[] args) {
        // Given array of names with duplicates
        String[] names = {"Aman", "Varchasv", "Sourabh", "Varchasv",
"Aman"};

```

```

// Using HashSet to store unique names
HashSet<String> nameSet = new HashSet<>();

// Adding names to HashSet (automatically removes duplicates)
for (String name : names) {
    nameSet.add(name);
}

// Check if a specific name exists
String checkName = "Varchasv";
if (nameSet.contains(checkName)) {
    System.out.println(checkName + " exists in the set.");
} else {
    System.out.println(checkName + " does not exist in the set.");
}

// Print the unique names
System.out.println("Unique names in the set:");
for (String name : nameSet) {
    System.out.println(name);
}
}
}

```

```

Varchasv exists in the set.
Unique names in the set:
Varchasv
Aman
Sourabh

```

5) Given an ArrayList of integers (possibly with duplicates), find the sum of its unique values. [Hint: Convert it to HashSet]

Example:

Input: [3, 5, 3, 8, 2, 5]

Unique values: [3, 5, 8, 2]

Sum: 18

```
import java.util.ArrayList;
import java.util.HashSet;

public class UniqueSum {
    public static void main(String[] args) {
        // Given ArrayList with duplicates
        ArrayList<Integer> numbers = new ArrayList<>();
        numbers.add(3);
        numbers.add(5);
        numbers.add(3);
        numbers.add(8);
        numbers.add(2);
        numbers.add(5);

        // Convert ArrayList to HashSet to remove duplicates
        HashSet<Integer> uniqueNumbers = new HashSet<>(numbers);

        // Calculate sum of unique values
        int sum = 0;
        for (int num : uniqueNumbers) {
            sum += num;
        }

        // Display results
        System.out.println("Unique values: " + uniqueNumbers);
        System.out.println("Sum: " + sum);
    }
}
```

```
Unique values: [2, 3, 5, 8]
Sum: 18
```